

GROUP NO. 10

DIGITAL VOTING SYSTEM

Object Oriented Programming — Project-Based Learning
Vidya Academy of Science And Technology

Java

JDBC

SQLite

TEAM — GROUP NO. 10

- | | | |
|---|--------------|--------------|
| 1 | Sradha | Roll No. 234 |
| 2 | Sreedevi | Roll No. 235 |
| 3 | Sreehari K N | Roll No. 236 |
| 4 | Sreehari N S | Roll No. 237 |



Java • JDBC • SQLite

WHAT IS THE DIGITAL VOTING SYSTEM?

The **Digital Voting System** is a Java-based application designed to provide a structured and secure electronic voting workflow. It replaces paper registers and manual tallying with a database-backed process in which every user must authenticate before reaching the functions that belong to their role.

The system supports secure authentication, role-based access for Admin, Candidate and Voter, candidate management, voter management, vote casting, duplicate-vote prevention, automatic vote counting, candidate ranking and audit logging. Each capability is implemented as a dedicated operation, which keeps the workflow easy to follow, test and extend.

Electronic voting is useful because it reduces manual work, speeds up counting, reduces human errors, provides structured records and makes the voting workflow easier to manage. The project is built with **Java**, **JDBC** and **SQLite**; according to the project specification, no external frameworks are required.

TECHNOLOGY

Java

JDBC

SQLite

No external frameworks required.

CORE CAPABILITIES

Secure
Authentication

Role-Based
Access

Candidate
Management

Voter
Management

Vote Casting

Duplicate-Vote
Prevention

Automatic Vote
Counting

Candidate
Ranking

Audit Logging

WHY ELECTRONIC VOTING?

- **Reduces manual work** — no paper registers to maintain.
- **Speeds up counting** — totals come straight from the database.
- **Reduces human errors** — fewer manual entries and tally mistakes.
- **Provides structured records** — users, votes and actions are stored consistently.
- **Easier to manage** — one workflow for every role.

CHALLENGES IN MANUAL VOTING

1

Time-consuming process

Manual registration, ballot handling and counting require significant time.

2

Human errors

Manual recording and tallying can result in incorrect entries or counting mistakes.

3

Duplicate voting and identity risks

A manual process requires reliable verification to ensure that each eligible voter votes only once.

4

Security and record-keeping

Maintaining reliable and tamper-resistant records is difficult when the process is completely manual.

HOW THIS PROJECT SOLVES IT

- **Automated authentication** — passwords are hashed with SHA-256 and verified before access.
- **Database-backed records** — voter and candidate data live in SQLite, not paper registers.
- **Database-enforced uniqueness** — a UNIQUE rule on the voter identifier blocks second votes.
- **Atomic transactions** — a vote is stored completely or not at all.
- **Rollback support** — an error undoes partial changes.
- **Audit logging** — important actions are recorded for later review.
- **Automatic vote tallying** — totals and rankings are computed from stored votes.

MANUAL PROCESS

paper registers • hand counting

PROBLEMS

delay • errors • duplicates • weak records

DIGITAL SOLUTION

authentication • integrity • audit trail

1

Secure Authentication

The system stores hashed passwords and verifies users before allowing access to role-specific functions.

2

Prevent Duplicate Votes

Duplicate voting is prevented at both the application level and the database level.

3

Easy Vote Management

The administrator can add candidates and voters through the application.

4

Role-Based Access

Separate functionality is provided for Admin, Candidate and Voter users.

5

Accurate Results

Votes are counted from the database and candidate rankings can be generated automatically.

HOW THE OBJECTIVES WORK TOGETHER

Authentication decides who may enter the system, and role-based access decides what each user may do. Once a voter is allowed to vote, application-level and database-level checks keep the election fair, while the administrator manages voters and candidates. Finally, votes are counted from the database so the displayed results always reflect the stored records.

MANUAL VOTING



Manual records + slow counting + human error

- **Slow counting** — every ballot is tallied by hand.
- **Manual intervention** — many steps depend on people.
- **Human error** — mistakes can enter records and totals.
- **Difficult record maintenance** — paper records are hard to organise.

BASIC E-VOTING



Automation but potentially limited security and auditing

- **Limited automation** — only some steps are digital.
- **Weak security** — protection mechanisms may be minimal.
- **No audit trail** — activity may not be recorded.
- **Limited scalability** — designs may not grow easily.

OUR SYSTEM



Authentication + database integrity + transactions + audit trail

- **Authentication** — hashed passwords and role checks.
- **Database integrity** — uniqueness constraints in SQLite.
- **Transactions** — atomic voting with rollback.
- **Audit trail** — key actions recorded for review.

OUR APPROACH — WHY THESE CHOICES

SHA-256 hashing

no plain-text passwords

SQLite storage

persistent, file-based

JDBC

Java-to-database link

Atomic transactions

all-or-nothing votes

Rollback support

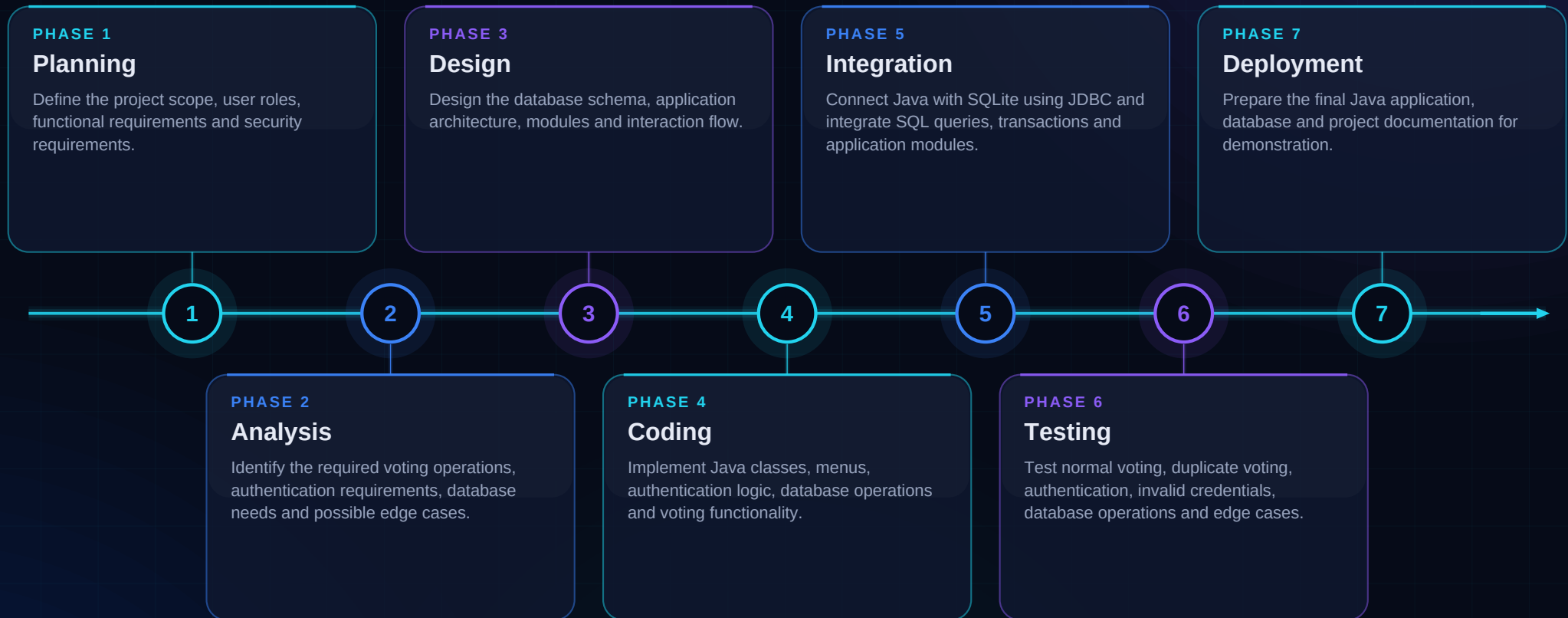
no partial updates

Audit logging

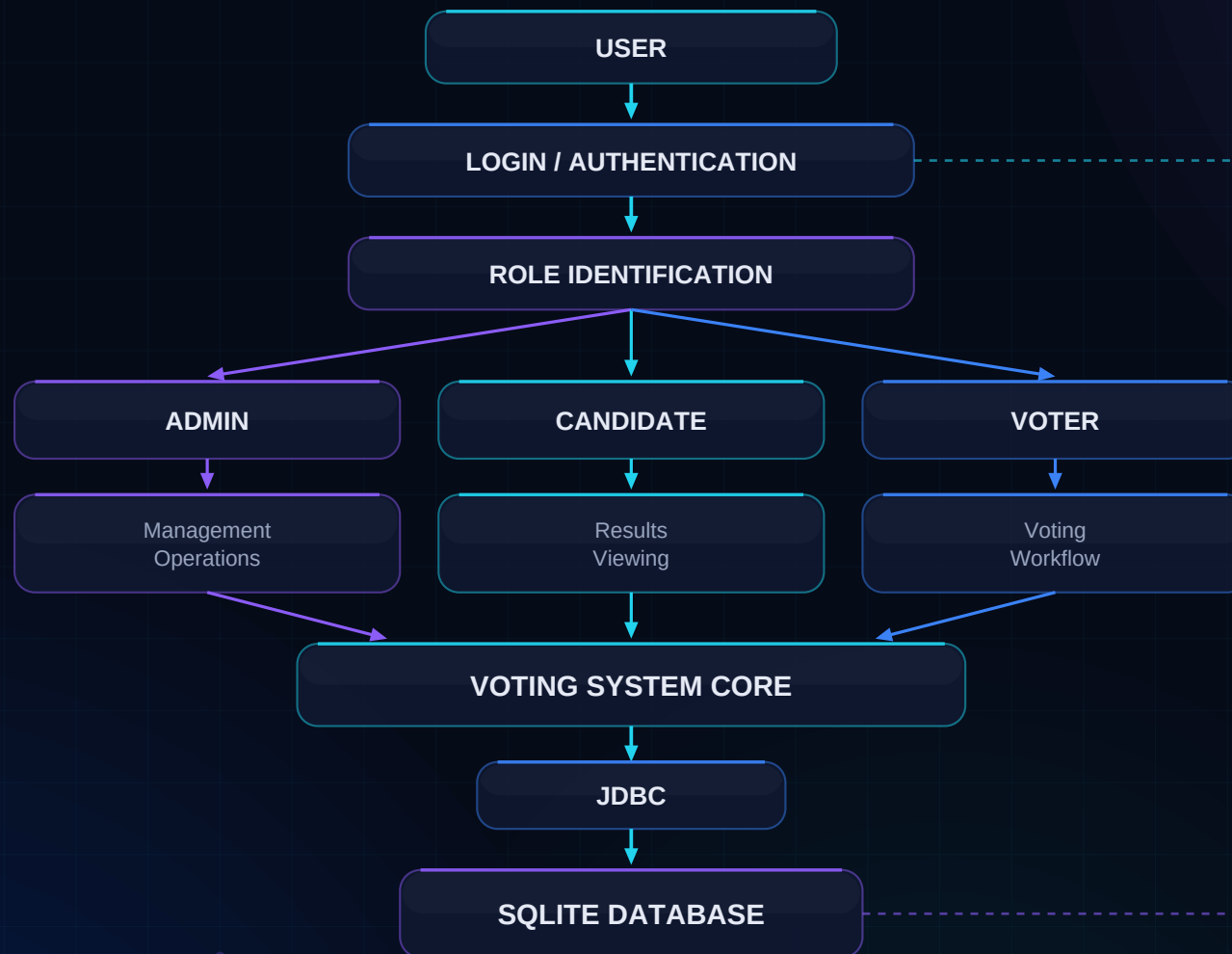
reviewable activity

UNIQUE constraints

one vote per voter



Iterative refinement. The phases are followed in order, but findings from testing may send the team back to design or coding so that problems are fixed before the final demonstration.



SECURITY

SHA-256

Passwords are converted into SHA-256 hashes during login so that credentials can be compared with stored digests and roles can be validated.

DATABASE



Users



Candidates



Votes



Audit Log

The architecture is layered. A user must first pass authentication; the identified role decides which workflow opens. All three workflows use the central **VotingSystem** core, which reaches the SQLite database only through JDBC prepared statements and transactions. Security (SHA-256 hashing) protects the login stage, while the database holds users, candidates, votes and the audit log.

CLASS

VotingSystem

VotingSystem is the central class responsible for encapsulating the major system operations such as login, voting and result calculation.

OBJECTS

Scanner, Connection, PreparedStatement

These objects support **input handling** (Scanner) and **database interaction** (Connection and PreparedStatement) during program execution.

ENCAPSULATION

Operations grouped in methods

Database and login-related operations are grouped into methods within the application classes rather than being exposed directly to the user interface.

ABSTRACTION

Methods hide the internals

Separate methods represent specific operations — **login, vote, results, database initialization and authentication**. Users interact with them without needing to know the database details.

INHERITANCE / POLYMORPHISM

NOT USED IN THIS VERSION

Inheritance and polymorphism are **not used** in the current version. The project focuses on **modular programming, encapsulation and abstraction**, which are sufficient for its scope.

1 DATABASE

RESPONSIBLE FOR

- Connecting to SQLite
- Creating tables
- Executing queries
- Managing persistent data

2 SECURITY

RESPONSIBLE FOR

- Password hashing
- Authentication support

3 ADMIN

RESPONSIBLE FOR

- Adding candidates
- Adding voters
- Viewing system information
- Managing administrative operations

4 CANDIDATE

RESPONSIBLE FOR

- Candidate-related access
- Viewing result information

5 VOTER

RESPONSIBLE FOR

- Viewing candidates
- Casting a vote
- Preventing repeated voting

6 RESULTS

RESPONSIBLE FOR

- Counting votes
- Displaying candidate totals
- Generating rankings

METHOD	PURPOSE
<code>connect()</code>	Establishes database connectivity
<code>initDatabase()</code>	Creates / initializes the database structure
<code>hash()</code>	Converts passwords into SHA-256 hashes
<code>auth()</code>	Validates user credentials and role
<code>login()</code>	Handles the login workflow
<code>manage()</code>	Performs management operations
<code>vote()</code>	Records a voter's vote
<code>tally()</code>	Calculates voting results

These eight methods split the major responsibilities of the application into small, focused operations — connecting and preparing the database, securing and verifying credentials, running the login and management workflows, recording votes and calculating results. Keeping each responsibility in its own method makes the code easier to read, test and maintain.

J

JAVA

JDK 8+

Primary programming language used to implement application logic and demonstrate OOP concepts.

S

SQLITE

File-based database

A lightweight, file-based relational database. It needs no separate database server, which suits a compact academic application.

J

JDBC

Java Database Connectivity

Connects Java code to SQLite and executes SQL statements, prepared statements and transactions.

E

ECLIPSE IDE

Development environment

Used as the development, debugging and build-management environment according to the project specification.

G

GIT

Optional version control

Optional version-control technology used to track source-code changes.

FUTURE TECHNOLOGY / POSSIBLE EXTENSION

SPRING BOOT

Spring Boot is **not part of the current implementation**. It is listed only as a possible future extension, for example to expose the voting logic through a web or REST interface.

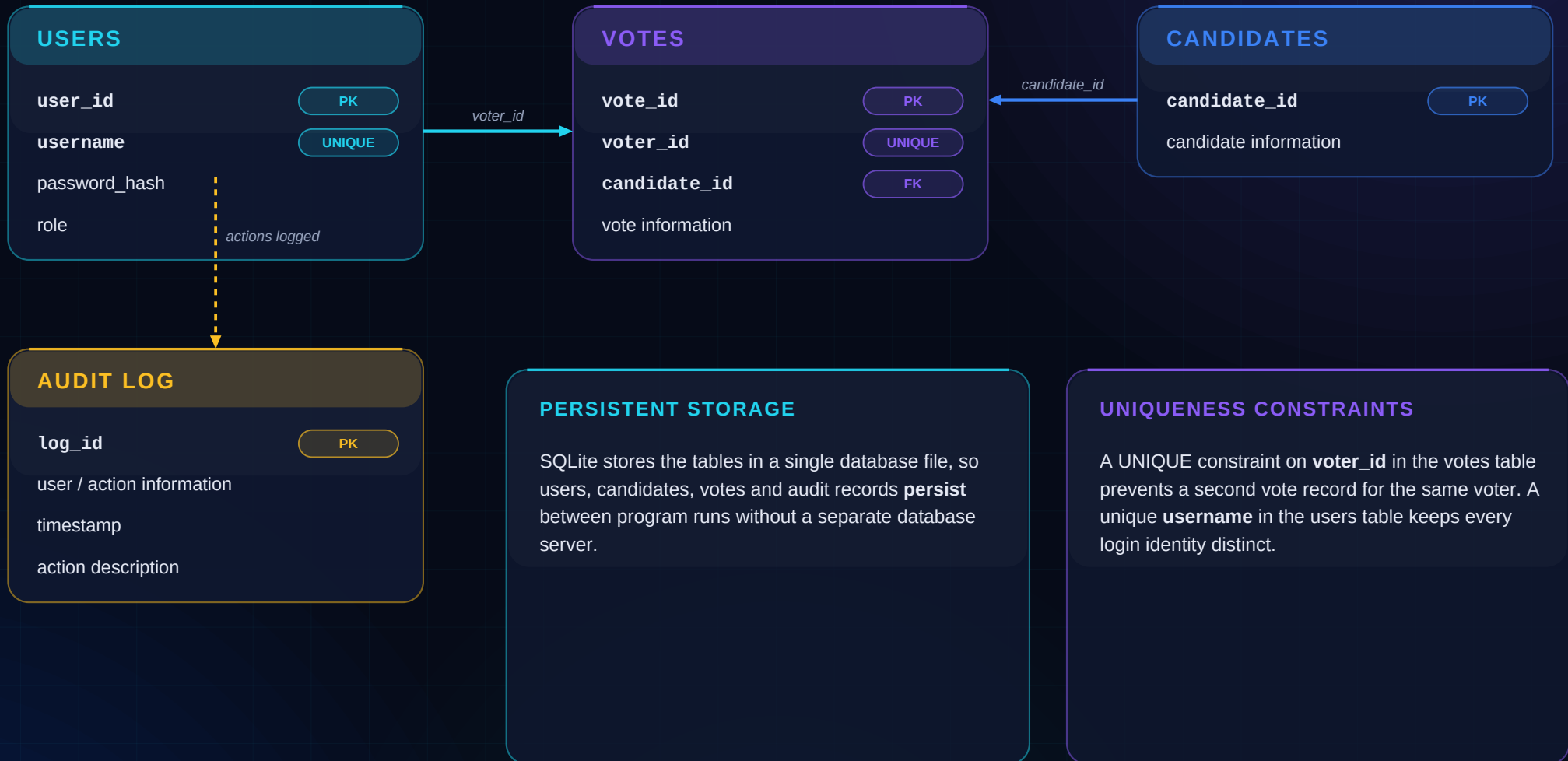
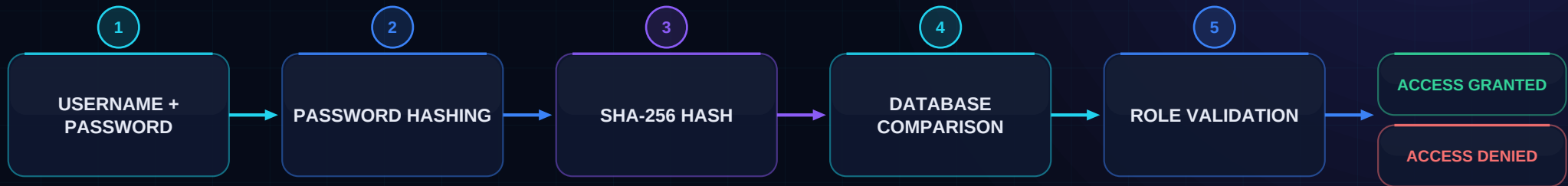


Table and column names are conceptual and show the logical structure of the schema.



LOGIN PROCESS EXPLAINED

The user supplies a username, password and role. The password is converted into a **SHA-256 hash**, and that digest — not the typed text — is compared with the value stored in the database. Passwords are not directly compared as plain-text stored values in the intended security model.

If the credentials match, the stored role is validated against the requested role, and the matching dashboard (Admin, Candidate or Voter) opens. Any mismatch results in access being denied.

Why SHA-256? It is the hashing algorithm specified by the academic project, and it keeps the example simple and easy to demonstrate.

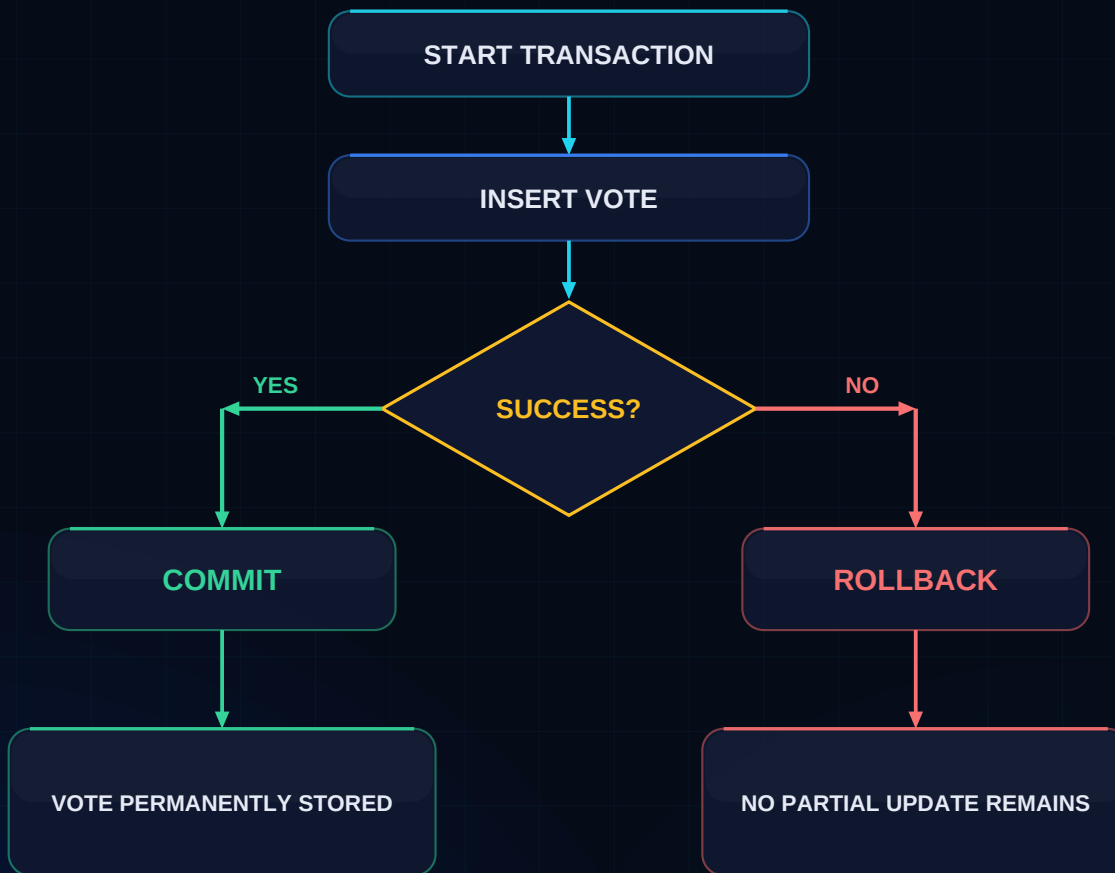
PRODUCTION IMPROVEMENT

For a real production system, **password-specific key derivation algorithms** such as **Argon2**, **bcrypt** or **scrypt** would be preferable to a plain SHA-256 digest, because they are deliberately slow and resist brute-force attacks.

The current application does **not** use them; this is noted as a possible hardening step.



Key control points: the voting status is checked *before* the candidate list is shown, the vote and its transaction are handled as one operation, and the final step ensures that the same voter cannot vote a second time.



HOW IT WORKS

The voting operation is treated as an **atomic database operation**: all of its steps succeed together, or none of them take effect.

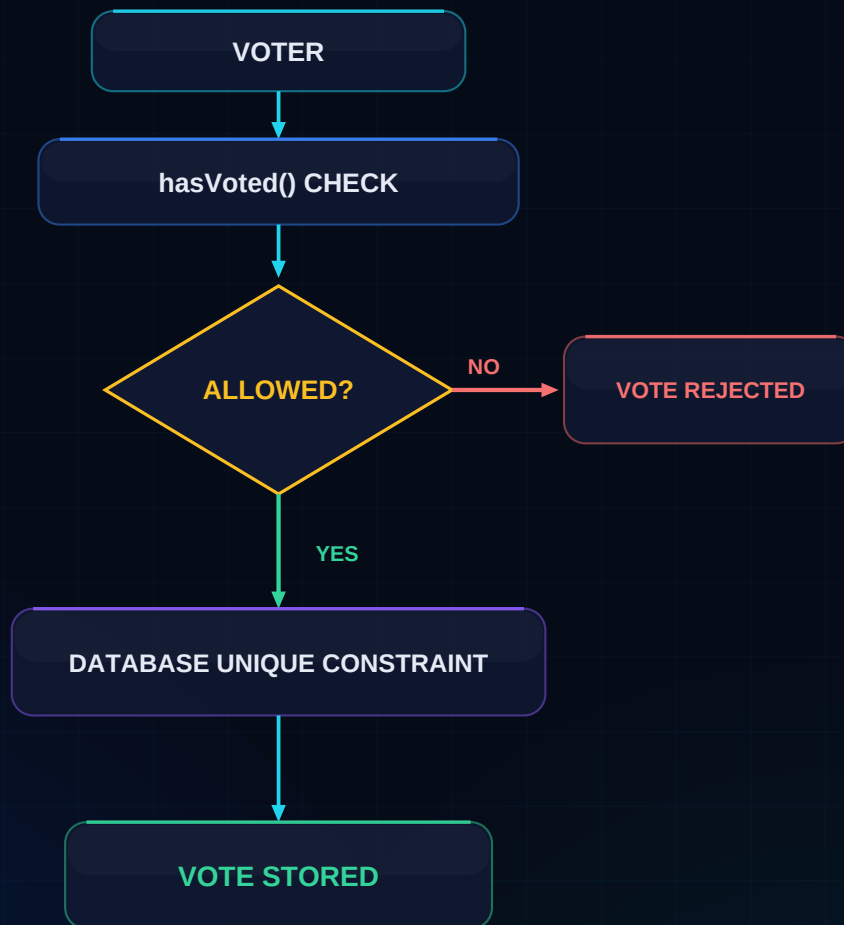
- **Auto-commit is disabled** for the voting operation, so changes are held until the program decides to commit.
- **The vote is inserted** into the votes table inside the open transaction.
- **On success, the transaction is committed** and the vote becomes permanent.
- **If an error occurs, rollback** prevents a partial database update.

DATABASE INTEGRITY

Because a failed vote can never leave half-written data behind, the votes table always stays consistent with what voters actually completed.

DUPLICATE VOTE PREVENTION

Two independent layers make sure that each voter votes only once



APPLICATION LEVEL

Before a vote is accepted, the system checks whether the voter has **already voted**. If so, the voter is stopped early with a clear result instead of reaching the database operation.

DATABASE LEVEL

A **uniqueness constraint on the voter identifier** in the votes table prevents another vote record for the same voter, even if the application check were bypassed or two requests overlapped.

WHY BOTH LAYERS?

The application check gives fast, friendly feedback; the database constraint is the final safeguard that cannot be skipped. Together they are stronger than either one alone.

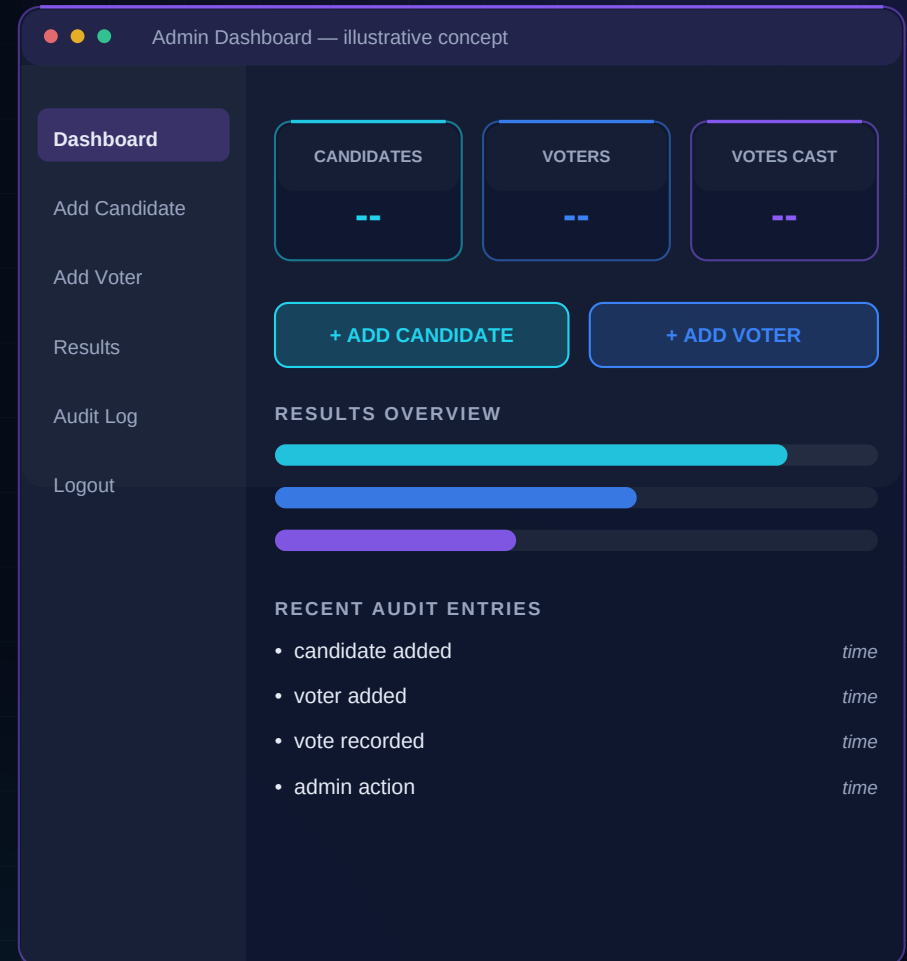
THE ADMINISTRATOR CAN

- **Add candidates** to the election.
- **Add voters** so that they can log in and vote.
- **View voting results** and candidate rankings.
- **Inspect audit information** about important actions.
- **Manage important system data** through administrative operations.

Administrative functionality is **separated from the voter workflow**. Role validation at login means that ordinary voters never reach these operations, which prevents them from modifying election information.

ROLE: ADMIN ONLY

Administration is performed through the same Java application used by every role; only the dashboard that opens differs.



WHAT THE CANDIDATE ROLE PROVIDES

The Candidate role is designed primarily for **viewing current result information**. A candidate logs in with credentials, is authenticated like every other user, and is taken to a results view.

- **Candidate login** with a username and password.
- **Authentication** against the stored SHA-256 hash.
- **Result viewing** from the database.
- **Current vote totals** for each candidate.
- **Candidate ranking** generated from the totals.

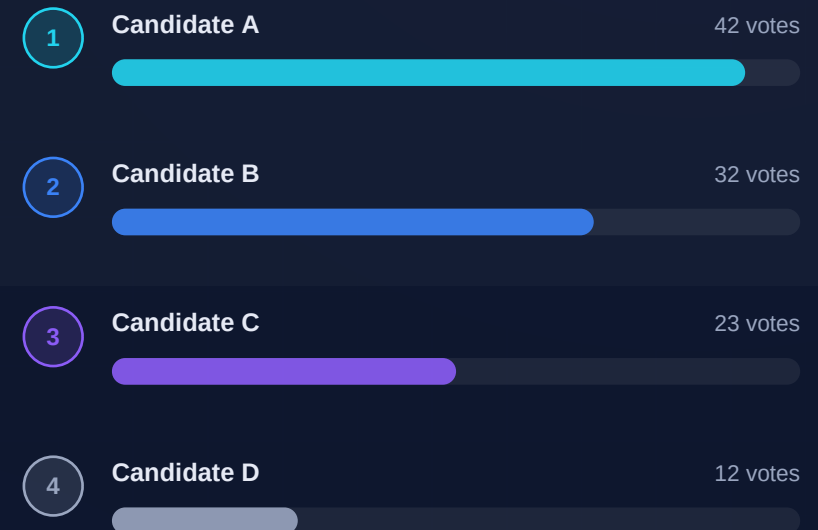
SCOPE NOTE

The role is limited to access and result viewing. Managing voters or candidates remains the responsibility of the Admin module.

Candidate Results View — illustrative concept

CURRENT STANDINGS

Sample data for illustration only



Ranking updates as votes are stored.

THE VOTER EXPERIENCE

- **Voter authentication** — username, password and the Voter role are verified.
- **Candidate list** — available candidates are displayed.
- **Candidate selection** — the voter picks one candidate.
- **Vote confirmation** — the choice is confirmed before it is stored.
- **Duplicate-vote prevention** — the voter cannot vote a second time.

This module is intentionally short: the voter sees only what is needed to vote. Checking the voting status first ensures that someone who has already voted is not shown the ballot again.

Voter Screen — illustrative concept

CAST YOUR VOTE

Select one candidate and confirm.

☐ Candidate A

☒ Candidate B

☐ Candidate C

☐ Candidate D

CONFIRM VOTE

CANCEL



```
SELECT candidate, COUNT(*) FROM votes GROUP BY candidate ORDER BY COUNT(*) DESC;
```

conceptual query

WHAT THE SYSTEM CALCULATES

- **Total votes** — the number of vote records stored in the database.
- **Votes received by each candidate** — votes are grouped by candidate and counted.
- **Candidate ranking** — candidates are ordered from highest to lowest total.

Because every figure is derived from the votes table, the displayed results always match the stored records instead of relying on a separately maintained counter.

RANKING VISUAL

Illustrative sample data



WHY AUDIT LOGGING MATTERS

The system records important operations so that activity can be **reviewed later**. The audit log provides an **audit trail for accountability**, helping administrators understand who did what, and when.

EXAMPLES OF RECORDED ACTIVITY

Candidate creation

Voter creation

Voting activity

Administrative actions

SCOPE NOTE

The audit log is an **accountability trail** stored in the database. Cryptographic tamper-proofing is **not** part of the current implementation, so the log should be understood as a review aid rather than an unforgeable record.

audit_log — illustrative view

LOG ID	ACTION	TIME
1	Candidate added	timestamp
2	Voter added	timestamp
3	Vote recorded	timestamp
4	Vote recorded	timestamp
5	Results viewed	timestamp
6	Admin action	timestamp

TEST CASE	EXPECTED RESULT	TYPE
Correct login	User reaches correct dashboard	NORMAL
Wrong password	Login rejected	EDGE
Wrong role	Access rejected	EDGE
First vote	Vote successfully recorded	NORMAL
Second vote	Duplicate vote rejected	EDGE
Add candidate	Candidate stored	NORMAL
Add voter	Voter stored	NORMAL
Result calculation	Correct totals displayed	NORMAL
Database failure	Transaction rolled back	EDGE
Audit action	Activity recorded	NORMAL

Testing should cover both **normal operation** and **edge cases**. A good test pass confirms that valid actions succeed, invalid or repeated actions are rejected cleanly, and that stored data stays consistent afterwards.

TEST COVERAGE

- **Authentication** — valid and invalid credentials.
- **Authorization** — correct dashboard per role.
- **Duplicate votes** — second vote rejected.
- **Transactions** — a failure rolls back.
- **Results** — counts and ranking update.
- **Database** — records persist correctly.

**Sradha**

Roll No. 234

CONTRIBUTION

- Testing
- Documentation
- Presentation

**Sreedevi**

Roll No. 235

CONTRIBUTION

- Admin Module
- Candidate Module

**Sreehari K N**

Roll No. 236

CONTRIBUTION

- Database Design
- Schema Creation

**Sreehari N S**

Roll No. 237

CONTRIBUTION

- Voter Module
- Security Features

COLLABORATIVE DEVELOPMENT

The work was divided by module so that database, security, voter, admin/candidate and quality-assurance responsibilities could progress in parallel, with testing and documentation bringing the final result together.

CONCLUSION

The Digital Voting System demonstrates how Object Oriented Programming, JDBC and SQLite can be combined into a structured and secure electronic voting workflow. Hashed-password authentication, role-based access, duplicate-vote prevention, atomic transactions, automatic tallying and audit logging work together to address the main weaknesses of a manual process.

SECURE

OBJECT-ORIENTED

DATABASE-DRIVEN

TRANSACTIONAL

EXTENSIBLE

Built on Java, JDBC and SQLite with no external frameworks, it also forms a clear base for later improvements.

FUTURE ENHANCEMENTS

- **Spring Boot REST API** to expose the voting logic over HTTP.
- **Web frontend** with authentication.
- **Stronger password hashing** with Argon2, bcrypt or scrypt.
- **HTTPS** for secure transport.
- **Server database** for larger deployments.

THANK YOU

Group No. 10 • Sraddha 234 • Sreedevi 235 • Sreehari K N 236 • Sreehari N S 237

Vidya Academy of Science And Technology